

Object-Oriented Programming

Lab 1 — From Structured to Object-Oriented Thinking

Managing a Student Record System

Total Marks: 10 (Task 1: 3 — Task 2: 3 — Task 3: 4)

Submission: Source file(s) + written reflections as code comments

Overview. Today you will build the *same* system three times, each time with a slightly better tool. You start with raw parallel arrays in C, then group them into a better data structure, and finally move everything — data *and* behaviour — into a **class** in an OOP language of your choice. Each step fixes a real pain point from the step before. By the end you will have discovered, through your own frustration, *why* object-oriented programming exists.

Scenario

You are asked to build a **Student Record System** for a small course. The system must keep track of up to **50 students**, each with:

- a **name** (at most 50 characters), and
- an integer **mark** (0–100).

The required operations are the same across all three tasks:

Operation	Description
ADD	Insert a new student (name + mark). Print an error if the list is full.
REMOVE	Remove the student at a given position. Shift all later entries left.
UPDATE	Change the mark of the student at a given position.
DISPLAY	Print every student's position, name, and mark.
SEARCH	Find and print the first student whose name matches a query; report if not found.
TOP-N	Print the top n students by mark (descending). Do <i>not</i> modify the original list; sort a copy.

▷ Notice

Constraint for all tasks: use fixed-size arrays only. Do *not* use linked lists, dynamic memory allocation, pointers-to-pointers, or any built-in collection class (e.g. `ArrayList`, `List<T>`).

Task 1 Parallel Arrays in C**[3 marks]****What you will do**

Implement all six operations using two **parallel arrays** — one for names, one for marks — and a separate integer that tracks how many students are currently stored. Every function must receive these three items as parameters.

Starter skeleton

```
1  #include <stdio.h>
2  #include <string.h>
3
4  #define MAX 50
5  #define NLEN 50
6
7  /*
8   * Two parallel arrays + a count variable.
9   * Declare them in main() and pass them to every function.
10  * Do NOT make them global.
11  */
12
13 void add      (char names[][NLEN], int marks[], int *count,
14               const char *name, int mark);
15
16 void removeAt(char names[][NLEN], int marks[], int *count,
17               int index);
18
19 void update   (int marks[], int count,
20               int index, int newMark);
21
22 void display  (char names[][NLEN], int marks[], int count);
23
24 int  search   (char names[][NLEN], int count,
25               const char *query);
26
27 void topN     (char names[][NLEN], int marks[], int count,
28               int n);
29
30 /* ---- implement each function below ---- */
31
32 int main(void) {
33     char names[MAX][NLEN];
34     int  marks[MAX];
35     int  count = 0;
36
37     /* test your functions here */
38
39     return 0;
40 }
```

Listing 1: task1.c

Marking rubric

Criterion	Marks
add and display correct	0.5
removeAt shifts both arrays correctly	0.5
update and search correct	0.5
topN sorts a copy; original array unchanged	0.5
Written reflection (see below)	1.0
Total	3

Reflection — place answers in a comment block at the top of task1.c

- Count the parameters of `removeAt`. Does this feel natural? What would happen if you needed a third field, say a `student_id`?
- In `removeAt`, how many separate shift operations did you write? Why two instead of one?
- If a colleague accidentally passed the wrong array to `update`, could the code detect it? What kind of bugs could silently occur?

▷ Notice

Notice that `names` and `marks` always travel together as a pair. Whenever you shift one during removal you must shift the other separately. They are logically *one thing* (a student), but the code treats them as two unrelated arrays. This is the exact pain point that Task 2 addresses.

Task 2 Grouping with struct in C

[3 marks]

Motivation

A **struct** lets you bundle related fields into a single named type. Instead of two parallel arrays you will have *one* array of **Student** records. Your function signatures will shrink, your shift loop will collapse to one operation, and moving a student around will feel like moving a single thing — because it now *is* one.

What is a struct? A **struct** groups variables of different types under one name. After writing `Student s;` you access its fields as `s.name` and `s.mark`. Assigning `list[i] = list[i+1]` copies *both* fields at once — the compiler handles it. This is the first step toward **encapsulation**: keeping related data together.

Starter skeleton

```
1  #include <stdio.h>
2  #include <string.h>
3
4  #define MAX 50
5  #define NLEN 50
6
7  typedef struct {
8      char name[NLEN];
9      int mark;
10 } Student;
11
12 /*
13  * Every function now takes ONE array of Students
14  * instead of two separate arrays.
15  * Compare each signature carefully with Task 1.
16  */
17
18 void add      (Student list[], int *count,
19               const char *name, int mark);
20
21 void removeAt(Student list[], int *count, int index);
22
23 void update   (Student list[], int count,
24               int index, int newMark);
25
26 void display  (Student list[], int count);
27
28 int  search   (Student list[], int count, const char *query);
29
30 void topN     (Student list[], int count, int n);
31
32 /* ---- implement each function below ---- */
33
34 int main(void) {
35     Student list[MAX];
36     int count = 0;
37
38     /* test your functions here */
```

```

39
40     return 0;
41 }

```

Listing 2: task2.c

What changed from Task 1?

	Task 1	Task 2
Data layout	Two separate arrays	One array of Student
Shift in remove	Two separate shift operations	One: <code>list[i] = list[i+1]</code>
Function params	<code>names[], marks[], count</code>	<code>list[], count</code>
Copy for topN	Must copy both arrays	Copy one array
Add a new field	Change every function signature	Add one field to the struct

Marking rubric

Criterion	Marks
All six operations correct	1.5
<code>removeAt</code> uses a single-array shift	0.5
Written reflection (see below)	1.0
Total	3

Reflection — place answers in a comment block at the top of task2.c

- Compare the signature of `removeAt` in Task 1 and Task 2. What was removed and why?
- The **struct** keeps data together, but the functions (`add`, `display`, ...) are still floating around separately. Could someone call `update` with a completely wrong array by accident? Can the code prevent it?
- `main` can still write `list[0].mark = -999;` directly without going through any function. Is this a problem? What would prevent it?

What you have just discovered. Grouping data into a **struct** is the first half of encapsulation — it keeps related *state* together. But the functions that operate on that data are still detached, and the data is still fully exposed. The second half of encapsulation is locking the functions *and* the data together, and hiding the internals so outside code cannot corrupt them. That is exactly what a **class** does.

Task 3 Your First Class**[4 marks]****Motivation**

A **class** is the natural next step after **struct**. It bundles the data *and* the functions that operate on it into one unit, and it hides the internals so that outside code cannot accidentally corrupt the state.

	Task 2 (struct)	Task 3 (class)
Data location	Separate array declared in main	Private fields <i>inside</i> the class
Functions	Standalone; need <code>list[]</code> passed in	Member methods; access fields directly
Protection	Anyone can read or write any field	private fields block direct access
Calling display	<code>display(list, count)</code>	<code>roster.display()</code>

Part A — Implement the StudentList class**[3 marks]**

Design and implement a class called **StudentList** in your chosen language. Your class must satisfy all of the following requirements — this is your design.

1. Declare **private** fields to store the student names, their marks, and the current count. Choose appropriate types for your language.
2. Provide a **constructor** (Java) or a constructor method (C#) that initialises the count to zero.
3. Implement all six operations as **public** methods: **add**, **removeAt**, **update**, **display**, **search**, and **topN**, with the same behaviour described in the Scenario.
4. None of the six methods should need to receive the internal array or the count as a parameter — the object already owns them.
5. Write a **main** method that creates one **StudentList** object and exercises all six methods.

▷ Notice

You may not use any built-in collection class (`ArrayList`, `LinkedList`, `List<T>`, `Array`, etc.). Use a plain fixed-size array as a private field, just as you did in Tasks 1 and 2.

Part B — Write a second class entirely from scratch**[1 mark]**

Write a small stand-alone class called **Counter** in your chosen language. It must:

- store a single **private** integer value, initialised to 0 in the constructor;
- expose a public method `increment()` / `Increment()` that adds 1;
- expose a public method `reset()` / `Reset()` that returns the value to 0; and
- expose a public method `get()` / `Get()` that returns the current value.

Write a short **main** that creates a **Counter**, calls all three methods, and prints results to the console. This is the first class you have written entirely on your own — keep it.

Marking rubric — Task 3

Criterion	Marks
Class compiles; all data fields are <code>private</code> ; constructor initialises count	0.5
<code>add</code> , <code>display</code> , and <code>removeAt</code> correct	1.0
<code>update</code> , <code>search</code> , and <code>topN</code> correct	1.0
No method takes the internal array or count as a parameter	0.5
Part B: <code>Counter</code> compiles; all three methods behave correctly	1.0
Total	4

Final Reflection (ungraded — required for Lab 2 entry)

Before leaving, write three sentences (as comments in your Task 3 file or on paper) answering:

1. What concrete problem did replacing parallel arrays with a `struct` solve?
2. What concrete problem did replacing the `struct` and standalone functions with a `class` solve?
3. `StudentList` is currently just a list. What change to the `add` and `removeAt` methods would turn it into a **stack**? Into a **queue**? You do not need to implement this now — just describe it in one sentence each. You will implement it in Lab 2.

General Notes

- Compile Tasks 1 & 2 with `gcc -Wall -o task1 task1.c` (and similarly for `task2.c`).
- Java: compile with `javac StudentList.java` and run with `java StudentList`.
- C#: compile with `dotnet run` or `csc StudentList.cs && mono StudentList.exe`.
- You may use standard string functions (`strcmp/strcpy` in C; `String.Equals/string.Equals` in Java/C#).
- Partial credit is awarded. Submit whatever compiles, even if some methods are incomplete.
- **Lab 2 preview:** you will extend your `StudentList` class into a **Stack** and a **Queue** using the same fixed-size array. Write clean code now — you will reuse it.

*“Each task fixes the pain left by the previous one.
That pain is not a bug in the exercise — it is the lesson.”*
